

Sentinel Code Review Report

File: sample2_vulnerable.py | Language: Python | Analysis ID: validation_sample2_vulnerable.py
Report generated: 2026-08-26 15:43:12 UTC

Executive / PR Summary

Analyzed sample2_vulnerable.py (Python). Found 3 security issue(s), most notably Possible SQL Injection (string-built query) (Critical, A03: Injection). The Code Analysis Agent flagged 1 structural issue(s).

PR Verdict: Block

Overall Code Health / Quality Score

Overall Score	Security	Quality	Maintainability	Complexity
62/100	32/100	92/100	88/100	Low

Severity Breakdown

Critical	High	Medium	Low	Info
2	0	2	0	0

Findings

■ [Critical] Possible SQL Injection (string-built query)

Category: A03: Injection **Type:** Security **Location:** Line 22

Explanation: A SQL statement is concatenated with a variable, allowing attacker-controlled input to alter the query. (line 22)

Root cause: A SQL string is built by string concatenation, so any attacker-controlled value becomes part of the query grammar itself.

Recommended fix: Use parameterized queries / bound placeholders instead of building SQL with string concatenation or f-strings.

Best practice: Treat every user-supplied value as data, never as part of the query string

```
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

■ [Critical] Command Injection

Category: A03: Injection **Type:** Security **Location:** Line 28

Explanation: A shell command is executed, and if any part comes from user input this allows arbitrary command execution. (line 28)

Root cause: A shell command is constructed from data that can be influenced by a caller/user, and the shell is invoked to interpret it.

Recommended fix: Avoid shell=True / string commands; pass an argument list and validate/allow-list any user input.

Best practice: Prefer library APIs over shelling out; if a shell command is unavoidable, pass an argument list and never `shell=True` with untrusted input

```
subprocess.run(["ls", "-la", safe_dir], shell=False, check=True)
```

■ [Medium] Weak cryptographic algorithm

Category: A02: Cryptographic Failures **Type:** Security **Location:** Line 32

Explanation: MD5/SHA-1/DES are cryptographically broken for security-sensitive use — use SHA-256+ or bcrypt/Argon2 for passwords. (line 32)

Root cause: A cryptographically broken or deprecated primitive is used for a security-sensitive operation.

Recommended fix: Use SHA-256+ for hashing and bcrypt/Argon2 for password storage instead of MD5/SHA-1/DES.

Best practice: Use SHA-256 or better for integrity, and bcrypt/Argon2/scrypt (never a bare hash) for password storage

```
import hashlib
digest = hashlib.sha256(data).hexdigest()
```

■ [Medium] Bare except clause

Category: Code Quality **Type:** Code Quality **Location:** Line 39

Explanation: A bare `except:` at line 39 silently swallows every exception, hiding real bugs.

Root cause: The exception type is not specified, so the handler intercepts every exception — including ones it wasn't written to handle.

Recommended fix: Catch a specific exception type and log it instead of silencing every error.

Best practice: Catch the specific exception type you expect and handle or log it explicitly

```
try:
    do_work()
except ValueError as e:
    logger.exception(e)
```

Prioritized Remediation Roadmap

1. **[Critical] Possible SQL Injection (string-built query)** — Use parameterized queries / bound placeholders instead of building SQL with string concatenation or f-strings.
2. **[Critical] Command Injection** — Avoid `shell=True` / string commands; pass an argument list and validate/allow-list any user input.

Overall Code Quality / Security Assessment

Overall score 62/100. PR verdict: **Block**. 4 total finding(s) (2 critical, 0 high, 2 medium, 0 low).

Best practice suggestions:

- Use parameterized queries or an ORM query builder instead of string formatting for database operations.
- Load API keys and sensitive credentials from environment variables or a secure secrets manager.
- Avoid executing raw shell commands; use structured APIs and validate all input parameters against an allow-list.
- Use modern cryptographic algorithms (e.g. SHA-256 for hashing, bcrypt/Argon2 for passwords).
- Catch specific exception classes and log meaningful diagnostic details rather than suppressing errors.

Performance suggestions:

- Batch database calls where possible and ensure appropriate indexing to prevent N+1 query bottlenecks.
- Use generators, list comprehensions, and built-in algorithms for memory-efficient data processing.
- Utilize connection pooling and asynchronous I/O where applicable to maximize throughput.

Code smells: No significant code smells detected.

